SET 23

PART A — Short questions and answers

Q1. Explain how the 1:N relationship "Agent handles Parcel" is mapped into tables.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q2. Summarize the integrity constraints used to keep the data of a table correct.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q3. Explain why Agent_Name should be moved to a separate table to attain 3NF.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q4. Explain how a LEFT JOIN identifies agents who have no parcels.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q5. Explain how SUM(Charges) treats NULL values.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q6. Describe the purpose of the DECLARE section for variables and constants.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q7. Describe how nested tracking data of a parcel is stored in a document.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q8. Explain how dot notation is used to access an embedded field.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q9. Explain what the following MongoDB query returns: db.parcels.find({Status: "In Transit"}, {ParcelID: 1, _id: 0})

Answer: The query returns all documents that satisfy the specified filter condition.

Q10. Explain the purpose of the pattern (a)-[:ROUTE]->(b) in a MATCH clause.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

PART B — System solution

Scenario: A courier company maintains a Courier and Parcel Tracking System to store customer details, delivery

1(a) ER design:

• **CUSTOMER**(CustomerID, CustomerName, Phone, City); primary key: **CustomerID**.

• **COURIER_AGENT**(AgentID, AgentName, Zone, Phone); primary key: **AgentID**.

• **PARCEL**(ParcelID, CustomerID, AgentID, WeightKg, DestinationCity, DispatchDate, DeliveryStatus, Charges); primary key: **ParcelID**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

1(b) SQL DDL/DML: Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE CUSTOMER (
  CustomerID NUMBER PRIMARY KEY,
  CustomerName VARCHAR2(100),
  Phone VARCHAR2(100),
  City VARCHAR2(100)
```

```
);
CREATE TABLE COURIER_AGENT (
  AgentID NUMBER PRIMARY KEY,
  AgentName VARCHAR2(100),
  Zone VARCHAR2(100),
  Phone VARCHAR2(100)
);
CREATE TABLE PARCEL (
  ParcelID NUMBER PRIMARY KEY,
  CustomerID NUMBER,
  AgentID NUMBER,
  WeightKg NUMBER,
  DestinationCity VARCHAR2(100),
  DispatchDate DATE,
  DeliveryStatus VARCHAR2(100),
  Charges VARCHAR2(100)
);
```

2(a) Join query: Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM PARCEL x
JOIN CUSTOMER a ON x.CustomerID = a.CustomerID
JOIN COURIER_AGENT b ON x.AgentID = b.AgentID;
```

2(b) Aggregate query: Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

2(c) PL/SQL procedure: Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

3. MongoDB: Create the collections and insert documents using the fields listed in the set. Query with find() and the requested filter.

```
db.collection.insertMany([
  { _id: 1, name: 'Sample A', category: 'Example', price: 500 },
  { _id: 2, name: 'Sample B', category: 'Example', price: 300 }
]);
db.collection.find({ author: 'Specified Author' });
db.collection.find({ price: { $gt: 400 } });
```